

Основни типове данни. Списък

Твоят наръчник за управление на данни в Python

Защо ни трябва колекции от данни?

Когато програмираме, често трябва да запомним и обработим много данни наведнъж.

Хаос (Отделни променливи)



Ред (Списък)



Вместо да създаваме 100 различни променливи, използваме **Списък** – последователност от всякакви данни, съхранени на едно място.

Създаване и Отпечатване

Име на променливата

```
dogs = ['корги', 'лабрадор', 'пудел', 'джак ръсел']  
print(dogs)
```

Отпечатва целия списък в конзолата на един ред.

Квадратни скоби – знакът, че създаваме списък!

Елементите са разделени със запетая

Правило №1 на програмиста: Броим от НУЛА



Индекс = Пореден номер
на елемент в списъка.

В езиците за програмиране
индексирането винаги
започва от 0, а не от 1!



Достъп чрез Индекс

За да получим достъп до конкретен елемент, пишем името на списъка и слагаме неговия индекс в квадратни скоби плътно до него.

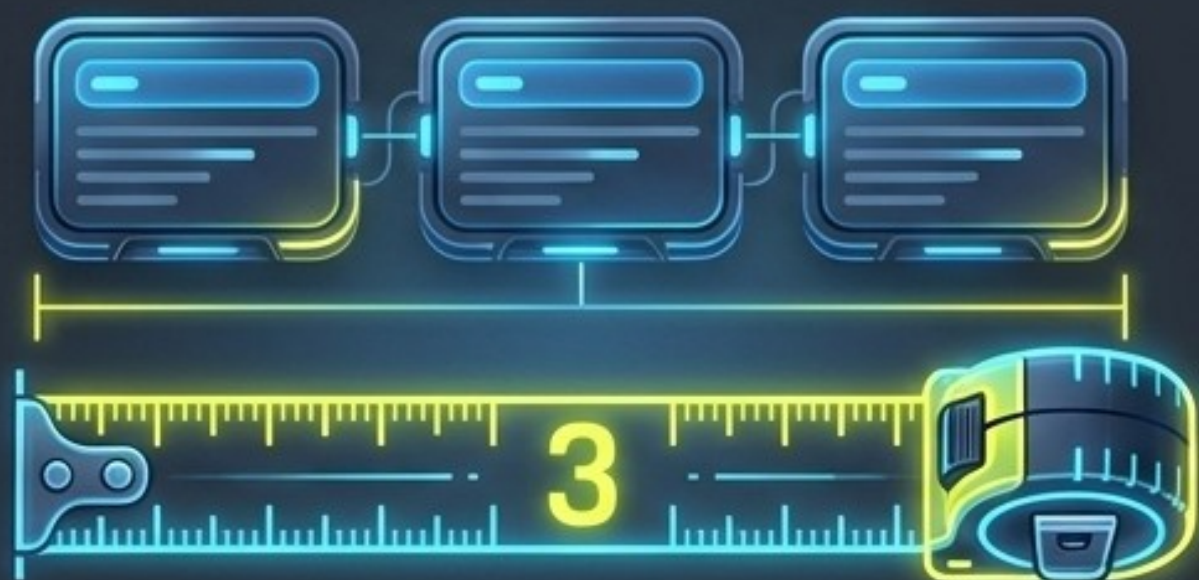


```
dogs[1] # Извежда 'лабрадор'  
dogs[0] # Извежда 'корги'
```



Дължина и празни списъци

len() – вградена функция, която връща броя на елементите.



```
nums = [2, 3, 4]
x = len(nums)
print(x) # Резултат: 3
```

Празен списък – готов за пълнене!



```
empty_list = []
empty_list = list()
```


Машината за поредици: range()

Командата range() създава автоматична поредица от последователни числа. Комбинираме я с list(), за да превърнем числата директно в списък!



```
list(range(2, 10)) -> Създава [2, 3, 4, 5, 6, 7, 8, 9]
```


ВАЖНО!!! Капанът на range()

Когато се задава интервал, началната стойност винаги е включена, а крайната е изключена!



```
x = range(5, 10)
```

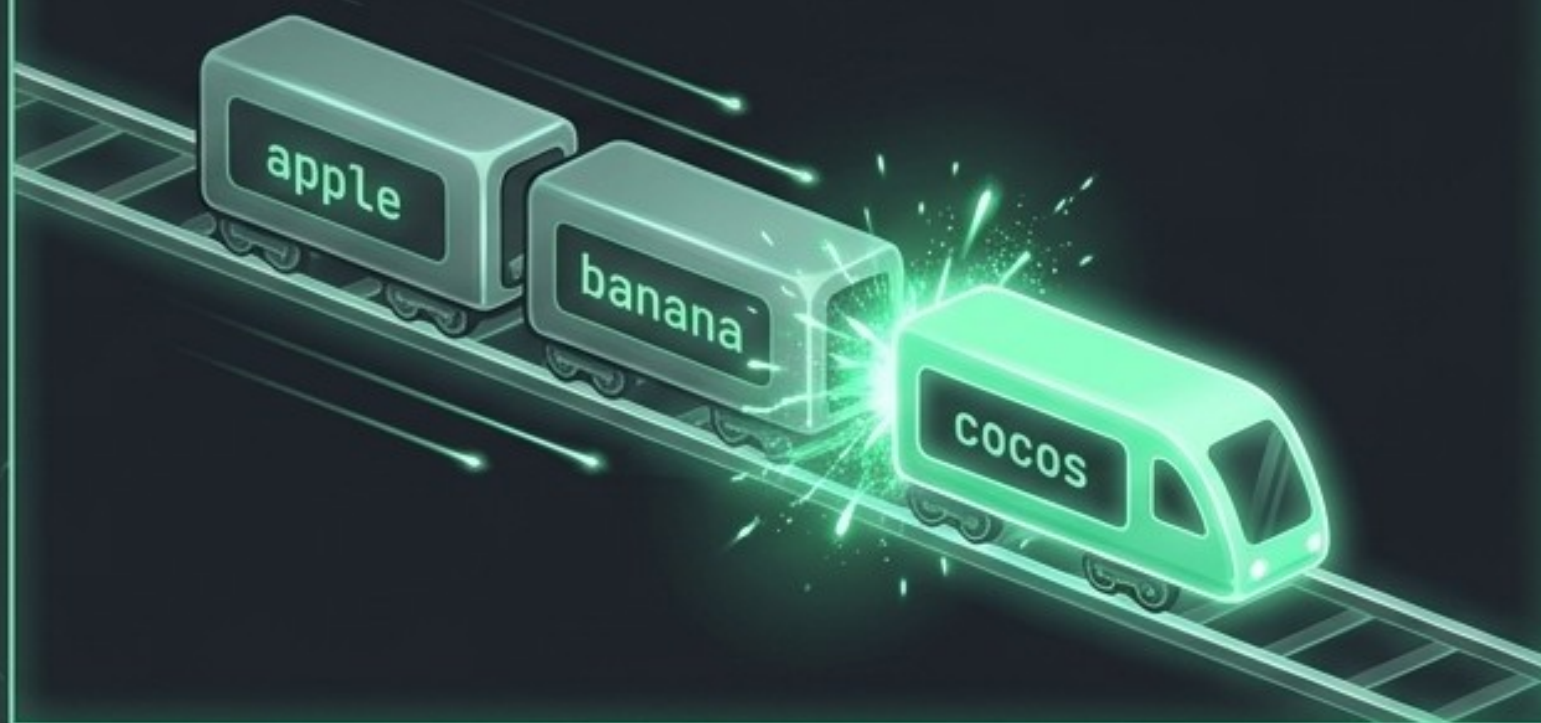
Резултат: 5, 6, 7, 8, 9 ...но НЕ и 10!

Забележка: Ако не зададем начало, по подразбиране започва от 0.

Управление на списъка: ДОБАВЯНЕ

append(стойност)

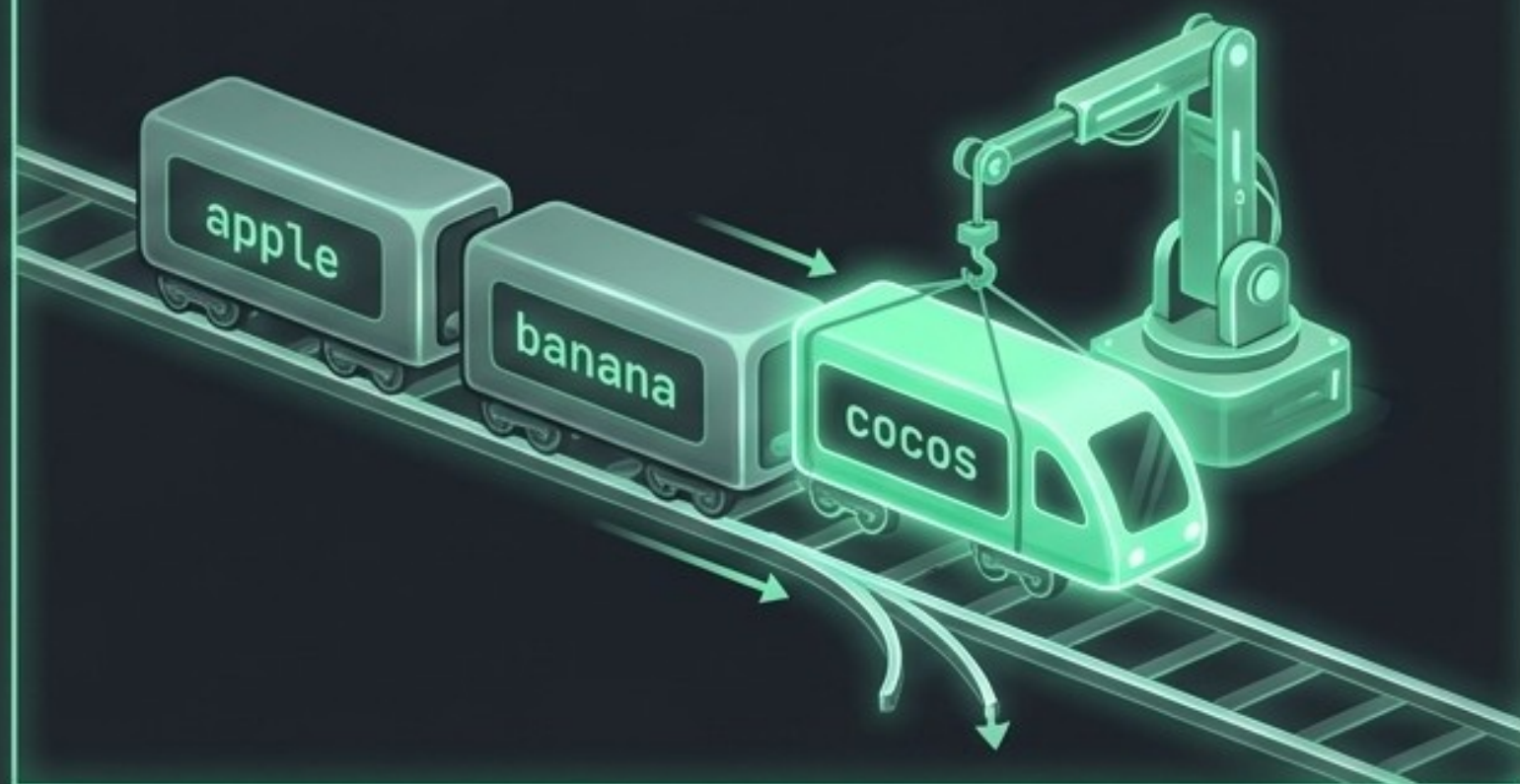
Добавя елемент в самия край на списъка.



```
fruits.append('cocos') -> ['apple', 'banana', 'cocos']
```

insert(индекс, стойност)

Вмъква елемент на точна позиция (индекс).



```
fruits.insert(0, 'cocos') -> ['cocos', 'apple', 'banana']
```


Управление на списъка: ПРЕМАХВАНЕ

remove(стойност)

Търси и изтрива първия намерен елемент по неговото име/стойност. Ако го няма – дава грешка!



```
colors.remove('blue')
```

pop(индекс)

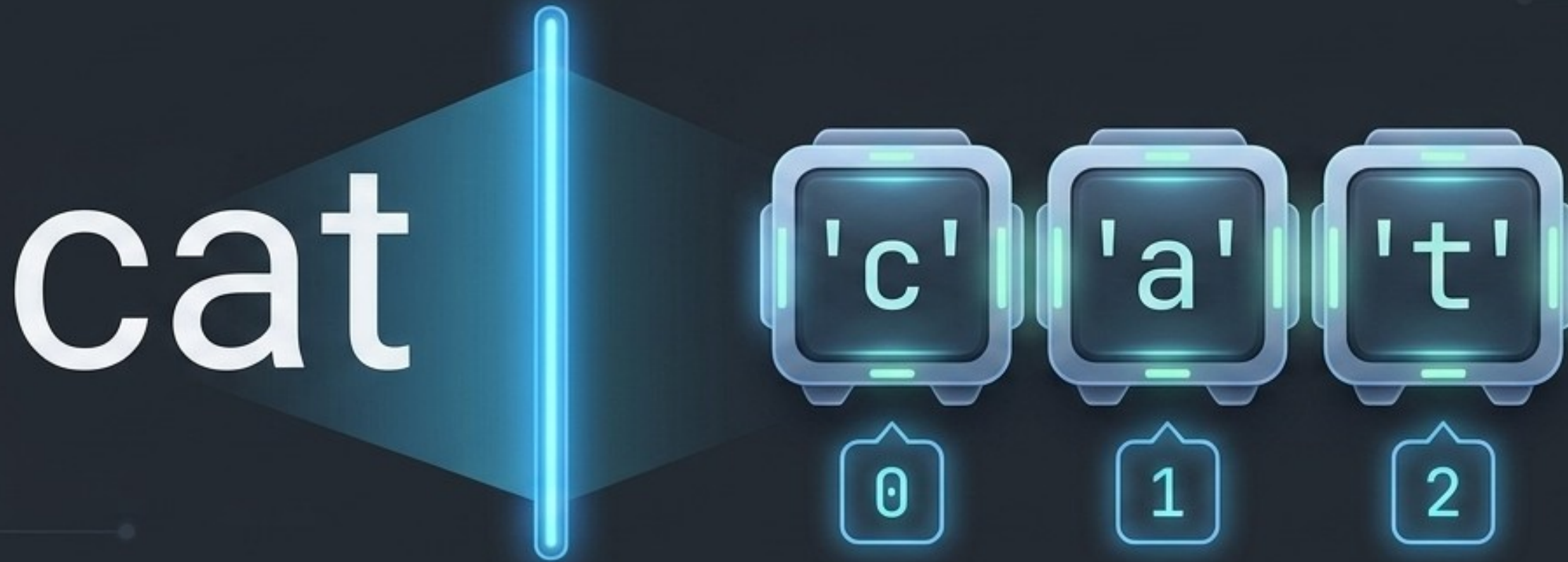
Изхвърля елемента, намиращ се на посочения индекс.



```
colors.pop(2)
```


Тайната на текста (Рентгенов изглед)

В Python всеки текст (string) всъщност е маскиран списък от символи! Имаме директен достъп до всяка буква чрез нейния индекс. Функцията `list()` може буквално да "нацепи" текст на букви.

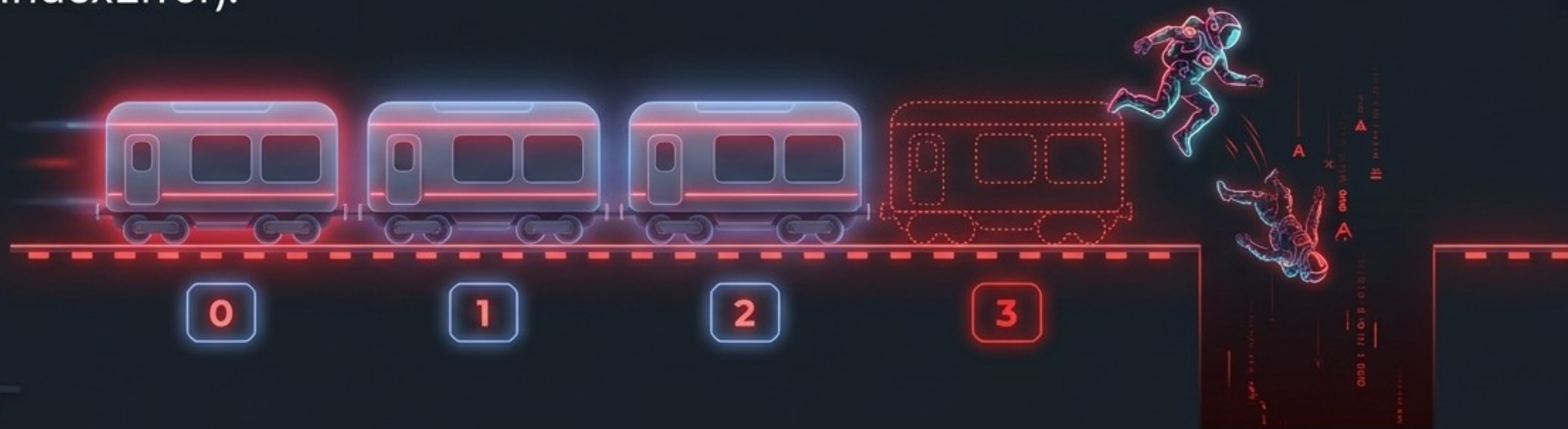


```
pet = 'cat'  
print(pet[0]) # Извежда 'c'
```

```
list1 = list('опасно!')  
# Резултат: ['о', 'п', 'а', 'с', 'н', 'о', '!', ' ']
```


Възможни проблеми: IndexError

Ако се опиташ да получиш достъп до индекс, който е по-голям от дължината на списъка, Python ще изведе съобщение за грешка (IndexError).



ПРАВИЛО: Индексът на последния елемент винаги е равен на `len(списък) - 1`.
Вагонът трябва да съществува, за да се качиш на него!

Пищовът на програмиста (Cheat Sheet)

Създаване

```
x = [1, 2, 3] или  
x = list()
```

Индекс

```
x[0] - първи елемент  
(Броим от 0!)
```

Магията Range

```
list(range(0, 5)) ->  
[0, 1, 2, 3, 4]
```

Дължина

```
len(x) - брой елементи
```

Добавяне

```
append(ст-ст) (в края)  
insert(инд, ст-ст) (на  
позиция)
```

Премахване

```
remove(ст-ст) (по име)  
pop(инд) (по позиция)
```